

Design Plan - Java O-O Program Visualizer/Animator

Team Members:

Prottasha Deb pdeb2024@my.fit.edu
Angelina Martis amartis2023@my.fit.edu
Arya Patel arya2023@my.fit.edu
Aiden Stensrud astensrud2024@my.fit.edu

Faculty Advisor:

David Luginbuhl dluginbuhl@fit.edu

Client:

David Luginbuhl dluginbuhl@fit.edu

Dates of Meetings with the Client for developing this Plan:

We plan to meet with the client bi-weekly on Wednesdays, starting on August 25th.

Goal and motivation:

Students in early Java courses often initially struggle with the concepts of classes and objects. These concepts can be hard to grasp because the inner workings of a program are both “invisible” to the programmer and occur rapidly during execution. A system that visualizes the internal memory of a Java program using human-comprehensible animation would be helpful for these new students, allowing them to more easily understand the Java language at a fundamental level.

Approach (key features of the system):

The Java O-O Visualizer is a tool for assisting novice coders in visualizing the inner workings of a Java program. Users can input a Java program into the visualizer. The visualizer then parses the Java program, determining how its instructions map to specific system memory instructions.

It then uses those system memory instructions to simulate the program's execution. At each step, the system memory is displayed to the user through an easy-to-understand memory diagram. As the program “executes,” the memory diagram will shift to reflect the machine's new state. This allows the user to see how the program’s memory is in its initial state and how it changes as a result of the program's instructions.

The memory diagram can display a lot of valuable information that teaches a new programmer exactly how the program works under the hood. It can show the creation and existence of variables, objects, classes, and object references. It can also demonstrate method calls, primitive type assignments, and parameter passing. It is also capable of depicting the visibility of fields and methods, as well as simple data structures such as arrays.

Novel features/functionalities:

A Java O-O visualizer had been attempted previously by a prior Senior Design group. Their attempt, however, fell short of a successful application due to a lack of focus on the memory diagrams and their accurate representation. Thus, we plan to overcome this shortcoming by emphasizing the accurate portrayal of the system memory through memory diagrams specifically, in a novel means of representing Java programs and their inner workings.

Algorithms and tools:

- Lexical analysis
- Syntax analysis
- [Memory diagrams](#)

Technical Challenges:

- While we plan to use common compiler strategies to parse the Java program into system memory instructions, we currently lack sufficient compiler knowledge and experience to do so.
- Our client has a very specific vision for how they would like the memory diagrams to appear visually. Thus, programmatically converting parser-derived memory instructions into visuals and animations that align with the client's vision will require considerable effort and trial and error.
- Since we plan to digitally animate all relevant low-level information in a given Java program, very complex Java programs could lead to convoluted, crowded memory diagrams, which will require efficient design for the visualizer to both handle and dynamically organize in order to keep the information comprehensible.

Milestone 1 (Sep 28): itemized tasks:

- Compare and select collaboration tools for software development, documents/presentations, communication, task calendar
- Compare and select the type of application to implement the Java visualizer as
- Compare and select technical tools/strategies for the memory instruction animator and the Java code parser
- Resolve technical challenge: Lack of compiler knowledge
- Design and demo a first draft for the appearance of the visualizer's output memory diagrams.
- Create Requirement Document
- Create Design Document
- Create Test Plan

Milestone 2 (Oct 26): itemized tasks:

- Implement, test, and demo the Java code parser
- Implement, test, and demo simple memory diagram generator
- Conceptualize, draft, and demo memory diagram animations
- Resolve technical challenge: Memory diagram design for complicated memory states

Milestone 3 (Nov 23): itemized tasks:

- Implement, test, and demo memory diagram animator
- Implement, test, and demo simple memory diagram generator for complex memory states
- Implement, test, and demo full cohesive application that combines Java code parser and memory diagram generator/animator
- Implement, test, and demo the user interface for the chosen type of application

Task matrix for Milestone 1:

Task	Arya	Aiden	Prottasha	Angelina
Resolve technical challenges	-	lack of compiler knowledge	-	-
Design memory diagram demo	design 50%	give input	give input	design 50%
Compare and select application type	web	-	-	desktop
Compare and select Collaboration Tools	presentation	programs	documents	communication, task calendar
Compare and select technical tools/strategies	-	java code parser	memory instruction animator	-
Requirement Document	write 10%	write 30%	write 50%	write 10%
Design Document	write 10%	write 50%	write 30%	write 10%
Test Plan	write 10%	write 30%	write 50%	write 10%

Approval from Faculty Advisor:

"I have discussed with the team and approve this project plan. I will evaluate the progress and assign a grade for each of the three milestones."

Signature: _____ Date: _____